

Resumen patrones

Patrones estructurales

Adapter

- **Aplicabilidad:** Usar adapter cuando quiero usar una clase existente y la interfaz no es compatible con lo que preciso.
- **Ventajas**
 - Una misma clase Adapter puede usarse para muchos Adaptees (el Adaptee y todas sus subclases).
 - El Adapter puede agregar funcionalidad a los adaptados.
- **Desventajas**
 - Se generan más objetos intermediarios.
- **Patrones relacionados**
 - **Decorator:** Decora objetos sin cambiar su interfaz, permite composición recursiva.
 - **Proxy:** Define un sustituto de otro objeto sin cambiar su interfaz.

Decorator

- **Aplicabilidad**
 - Agregar responsabilidades a objetos individualmente y en forma transparente (sin afectar otros objetos).
 - Quitar responsabilidades dinámicamente.
 - Cuando subclasificar es impráctico.
- **Ventajas**
 - Permite mayor flexibilidad que la herencia.
 - Permite agregar funcionalidad dinámicamente.
- **Desventajas**
 - Mayor cantidad de objetos.
 - Complejo para depurar.
- **Patrones relacionados**
 - **Adapter**
 - Ambos patrones “decoran” el objeto para cambiarlo.
 - El Decorator preserva la interfaz del objeto para el cliente.
 - Adapter convierte la interfaz del objeto para el cliente.
 - Decorators pueden y suelen anidarse.
 - Adapter no se anida.
 - **Composite**

- El decorator mantiene una estructura con sólo un siguiente.
- El composite mantiene una estructura de tipo árbol, un composite usualmente se compone de varias partes.
- El propósito del decorator es agregar funcionalidad dinámicamente.
- El propósito del composite es componer objetos y tratarlos de manera uniforme
- **Strategy**
 - Permiten que un objeto cambie su funcionalidad dinámicamente.
 - El Decorator cambia el algoritmo por fuera del objeto.
 - El Strategy cambia el algoritmo por dentro del objeto.

Proxy

- **Aplicabilidad:** Cuando se necesita una referencia más flexible hacia un objeto Aplicaciones del proxy:
 - **Virtual proxy:** demorar la construcción de un objeto hasta que sea realmente necesario, cuando sea poco eficiente acceder al objeto real.
 - **Protection proxy:** Restringir el acceso a un objeto por seguridad.
 - **Remote proxy:** Representa un objeto remoto en el espacio de memoria local. Es la forma de implementar objetos distribuidos. Estos proxies se ocupan de la comunicación con el objeto remoto, y de serializar/deserializar los mensajes y resultados.
 - Para acceder a objetos que se encuentran en otro espacio de memoria, en una arquitectura distribuida.
 - El proxy empaqueta el request, lo envía a través de la red al objeto real, espera la respuesta, desempaqueta la respuesta y retorna el resultado.
 - En este contexto el proxy suele utilizarse con otro objeto que se encarga de encontrar la ubicación del objeto real. Este objeto se denomina Broker, del patrón de su mismo nombre.
- **Ventajas**
 - Indirección en el acceso al objeto: El patrón Proxy introduce un nivel de indirección que permite controlar el acceso al objeto real, lo que ofrece flexibilidad en su uso.
- **Desventajas**
 - Complejidad adicional.

- **Patrones relacionados**

- **Adapter**

- El adapter proporciona una interfaz diferente al objeto que adapta.
 - El proxy proporciona la misma interfaz que su sujeto.

- **Decorator**

- El decorator agrega una o más responsabilidades a un objeto.
 - El proxy controla el acceso a un objeto.

Composite

- **Aplicabilidad:** Usar el patrón cuando:

- Quiera representar jerarquías.
 - Quiera que los objetos “clientes” puedan ignorar las diferencias entre composiciones y objetos individuales. Los clientes tratan a los objetos simples y compuestos uniformemente.

- **Ventajas**

- Define “jerarquías” de objetos primitivos y compuestos.
 - Los objetos primitivos pueden componerse en objetos complejos, los que a su vez pueden componerse recursivamente.
 - Simplifica los objetos cliente. Los clientes usualmente no saben (y no deberían preocuparse) acerca de si están manejando un compuesto o un simple.
 - Facilita agregar nuevos tipos de componentes porque los clientes no tienen que cambiar cuando aparecen nuevas clases componentes.

- **Desventajas**

- Puede resultar difícil proporcionar una interfaz común para clases cuya funcionalidad difiere demasiado. En algunos casos, tendrás que generalizar en exceso la interfaz componente, provocando que sea más difícil de comprender.

- **Patrones relacionados**

- **Decorator:** Suele usarse junto con el composite, normalmente ambos tendrán una clase padre en común. Por lo tanto, los decoradores tendrán que admitir component con operaciones como añadir, agregar y obtener hijo.

Patrones de creación

Builder

- **Aplicabilidad:** Usar el patrón cuando:
 - El algoritmo para crear un objeto complejo debiera ser independiente de las partes de que se compone dicho objeto y de cómo se ensamblan.
 - El proceso de construcción debe permitir diferentes representaciones del objeto que está siendo construido.
- **Ventajas**
 - Abstrae la construcción compleja de un objeto complejo.
 - Permite variar lo que se construye Director <-> Builder
 - Da control sobre los pasos de construcción
- **Desventajas**
 - Requiere diseñar e implementar varios roles.
 - Cada tipo de producto requiere un ConcreteBuilder.

Factory Method

- **Aplicabilidad:** Usar este patrón cuando:
 - Una clase no puede prever la clase de objetos que debe crear.
 - Una clase quiere que sean sus subclases quienes especifiquen los objetos que ésta crea.
 - Las clases delegan la responsabilidad en una de entre varias clases auxiliares, y queremos localizar qué subclase de auxiliar concreta es en la que se delega.
- **Ventajas**
 - Proporciona enganches para las subclases. Crear objetos dentro de una clase con un método de fabricación es siempre más flexible que hacerlo directamente.
 - Evita el acoplamiento entre el creador y los objetos concretos.
 - Aplica el principio OCP, permitiendo agregar nuevos tipos de productos sin modificar lo que ya funciona.
- **Desventajas**
 - Se puede complejizar el código si tienes que agregar muchos tipos de constructores.
- **Patrones relacionados**
 - **Builder**
 - El Factory Method delega la creación de objetos a sus clases concretas.
 - El Builder construye objetos complejos paso a paso a través de un director que ensambla las partes.

Patrones de comportamiento

Strategy

- **Aplicabilidad:** Usar patrón cuando:
 - Existen muchos algoritmos para llevar a cabo una tarea.
 - No es deseable codificarlos todos en una clase y seleccionar cuál utilizar por medio de sentencias condicionales
 - Cada algoritmo utiliza información propia. Colocar esto en los clientes lleva a tener clases complejas y difíciles de mantener.
 - Es necesario cambiar el algoritmo en forma dinámica, en tiempo de ejecución.
- **Ventajas**
 - Mejor solución que subclasificar el contexto, cuando se necesita cambiar dinámicamente.
 - Desacopla al *Context* de los detalles de implementación de las *Strategy*.
 - Elimina condicionales.
- **Desventajas**
 - La clase cliente debe conocer las diferentes estrategias para poder elegir.
 - Overhead en la comunicación entre *Context* y *ConcreteStrategy*

State

- **Aplicabilidad:** Usar patrón cuando:
 - El comportamiento de un objeto depende del estado en el que se encuentre.
 - Los métodos tienen sentencias condicionales complejas que dependen del estado.
 - Este estado se representa usualmente por constantes enumerativas y en muchas operaciones aparece el mismo condicional. El patrón State reemplaza el condicional por clases **(es un uso inteligente del polimorfismo)**
- **Ventajas**
 - Localiza el comportamiento relacionado con cada estado.
 - Las transiciones entre estados son explícitas.
 - En el caso que los estados no tengan variables de instancia pueden ser compartidos.
- **Desventajas**

- En general hay bastante acoplamiento entre las subclases de State porque la transición de estados se hace entre ellas, por lo que deben conocerse entre sí.
- **Patrones relacionados**
 - **Strategy**
 - En el Strategy necesito uno de diferentes algoritmos opcionales para realizar una misma tarea.
 - En el state el comportamiento de un objeto depende del estado en el que se encuentre.
 - El patrón Strategy es útil para permitir que una clase delegue la ejecución de un algoritmo a una instancia de una familia de estrategias
 - El patrón State es útil para una clase que debe realizar transiciones entre estados fácilmente.
 - El State es privado del objeto, ningún otro objeto sabe de él.
 - El Strategy suele setearse por el cliente, que debe conocer las posibles estrategias concretas.
 - Los states concretos se conocen entre sí. Saben a cual estado se debe pasar en respuesta a algún mensaje.
 - Los strategies concretos no.

Template Method

- **Aplicabilidad:** Usar patrón cuando:
 - Para implementar las partes invariantes de un algoritmo una vez y dejar que las subclases implementen los aspectos que varían.
 - Para evitar duplicación de código entre subclases.
 - Para controlar las extensiones que pueden hacer las subclases.
- **Ventajas**
 - Favorece el reuso de código.
- **Desventajas**
 - Algunas clases pueden verse limitadas por el template proporcionado.
 - Tienden a ser difíciles de mantener cuando el código escala.

Null object

- **Aplicabilidad:** Un objeto tiene un colaborador, que algunas veces no hace nada queremos que el objeto cliente pueda ignorar la

diferencia del colaborador que hace algo con el que no hace nada queremos **reusar el comportamiento de hacer nada**.

- **Ventajas**

- En cualquier lugar que el cliente espera un objeto real también puede tomar un null object
- Hace el código del cliente más simple. Los clientes pueden tratar a sus colaboradores de manera uniforme.
- Encapsula el comportamiento de hacer nada en un objeto, que puede ser reusado por múltiples clientes.

- **Desventajas**

- Podría ser difícil de implementar si la usan varios clientes y no hay un acuerdo de cuál debería ser el comportamiento nulo.

Refactoring to pattern

Form Template Method

- **Propósito del patrón**

- Definir el esqueleto de un algoritmo y diferir pasos a subclases.
- Permite redefinir pasos sin cambiar la estructura general.

- **Problemas que soluciona**

- Reduce/elimina código repetido en subclases.

- **Ventajas**

- Eliminar código duplicado.
- Simplifica y comunica el algoritmo.
- Las Subclases pueden adaptar fácilmente.

- **Desventajas**

- Complica el diseño si las subclases deben implementar muchos métodos.

Extract Adapter

Se aplica cuando tienes una clase que está haciendo "malabares" para comunicarse con otra que tiene una interfaz incompatible.

- **Motivación (El problema):** Una clase contiene lógica incrustada para traducir o adaptar llamadas hacia otra clase con una interfaz diferente. Esto viola el Principio de Responsabilidad Única (SRP), genera un acoplamiento innecesario y hace que el código sea muy difícil de testear.
- **Solución:** Extraer esa lógica de traducción y moverla a una clase **Adapter** completamente independiente.
- **Ventajas:** Disminuye el acoplamiento entre las clases y facilita la reutilización del código de adaptación. Mejora la testeabilidad al permitir simular el adapter (con Mocks o Stubs) y refuerza el principio SRP.

Move Embellishment to Decorator

"Embellishment" se refiere a "adornos" o comportamientos adicionales que se le van sumando a una clase, engordándola innecesariamente.

- **Motivación (El problema):** Clases que han crecido demasiado porque se les han ido agregando responsabilidades o "adornos" extras mediante lógica condicional o abuso de herencia.
- **Solución:** Mover esos comportamientos a objetos **Decorator** independientes. Esto permite añadir estas responsabilidades a un objeto de forma dinámica y completamente transparente.
- **Ventajas:** Otorga mucha más flexibilidad que la herencia, ya que permite añadir o quitar responsabilidades en tiempo de ejecución "envolviendo" a los objetos. Evita tener clases sobrecargadas y refuerza el SRP al dividir la clase en varias más pequeñas y cohesivas.

Replace Implicit Tree with Composite

Este refactoring se utiliza cuando estás construyendo estructuras jerárquicas (árboles) de forma manual y poco estructurada.

- **Motivación (El problema):** Ocurre cuando se construyen árboles utilizando primitivas (como concatenar **String** para generar XML o HTML). Esto acopla fuertemente la lógica con la representación. Como resultado, el código es propenso a errores y dificulta el mantenimiento y la extensión.

- **Solución:** Reemplazar esa construcción manual por un patrón **Composite**, utilizando objetos dedicados (por ejemplo, un **TagNode** para representar nodos XML).
- **Ventajas:** Mejora enormemente la comunicación de la estructura del código. Además, reduce la duplicación y los errores, facilitando el mantenimiento y futuras extensiones.

Replace Conditional Logic with Strategy

- **Motivación**
 - Cuando hay lógica condicional que elige variantes.
- **Qué hacer**
 - Crear un Strategy por variante.
 - Hacer que el método delegue en el strategy.

Replace State-Altering Conditionals with State

- **Problema**
 - Lógica condicional de cambio de estado compleja
- **Qué hacer**
 - Usar objetos State para manejar estados y transiciones.
- **Motivación**
 - Obtener una vista global de las transiciones.
 - Condicionales difíciles de mantener/extender.
 - Otros refactorings no alcanzan.

Introduce Null Object

- **Motivación (El problema):** Existe una alta cantidad de condicionales comprobando si un objeto es nulo. El objetivo es tratar al estado "nulo" como un objeto más en sí mismo, evitando así errores y excepciones por olvidos (el famoso **NullPointerException**).
- **Solución:** Crear un objeto nulo (**Null Object**) que comparta la interfaz del objeto real pero cuya implementación sea "no hacer nada".